Transactions

Learning Database



Transaction



Account A
200000

Transfer 50000



Account B
200000

1. Select amount from Account A
2. Update amount of Account A
3. Select amount from Account B
4. Update amount of Account B
5. Create transfer history data

Think about something is going wrong, while SQL executions …

Transaction

Business Application တွေရဲ့ Action တစ်ခု Database Execution တွေ တစ်ခုအထက်အများပါဝင်လေ့ရှိတယ်

ဥပမာအားဖြင့် Account တစ်ခုကနေ တစ်ခုကို ငွေလွှဲတယ်ဆိုပါစို့

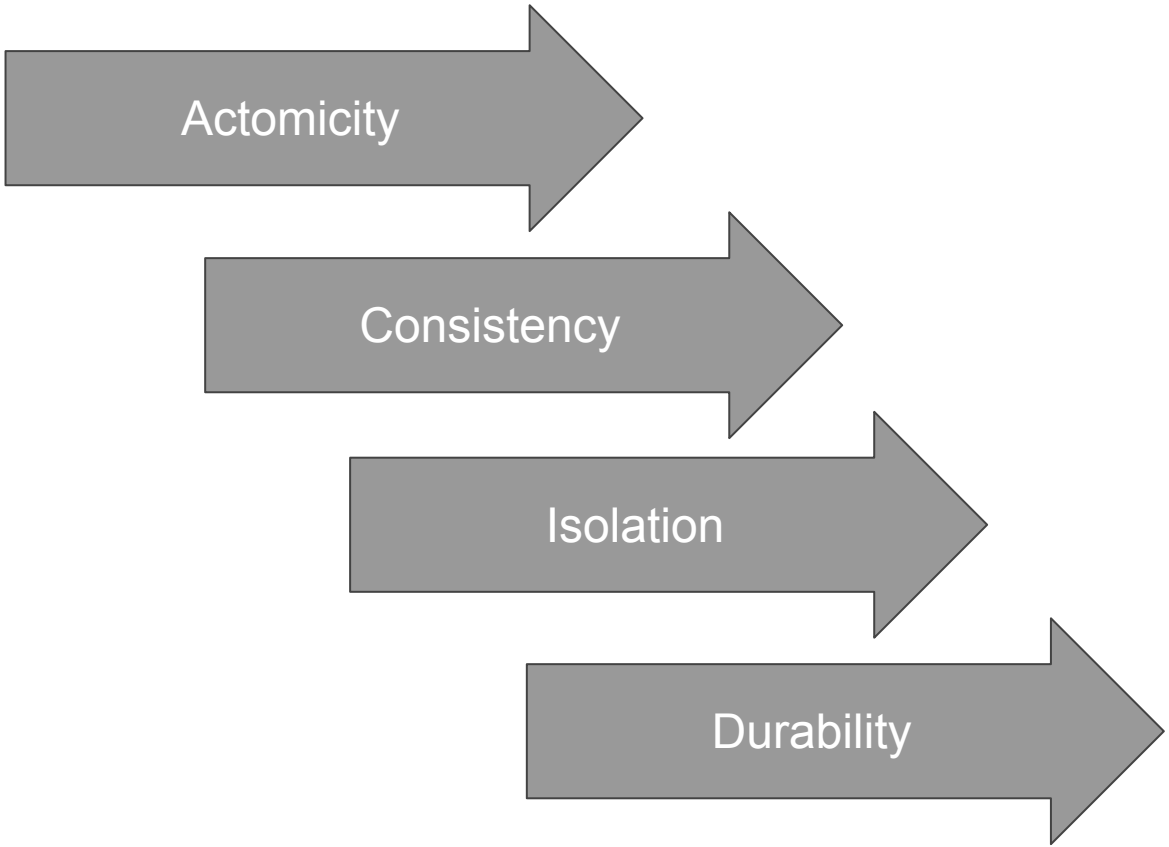
Account A ထဲကနေ ငွေကို နှုတ်မယ် ပြီးရင် Account B ထဲကို ပေါင်းမယ်။ လိုအပ်လာရင် သက်ဆိုင်ရာ History Table ထဲကိုလဲ Data တွေကို Insert လုပ်ဖို့လိုပါမယ်

အကြောင်းတစ်မျိုးမျိုးကြောင့် လုပ်ဆောင်နေစဉ် Error ဖြစ်တယ်ဆိုတာကို လက်ခံလို့မရပါဘူး

အဲဒီလို Business Action တစ်ခုအတွင်းမှာပါတဲ့ Database Execution တွေအားလုံးကိုစုစည်းပြီး Transaction တစ်ခုလို့သတ်မှတ်ပါတယ်

Requirements of Database Operations

ACID



Atomicity

Transaction တစ်ခုလုံး အောင်မြင်စွာပြီးဆုံးဖို့ လိုအပ်တယ်

လုပ်ဆောင်မှုတွေအားလုံးပြီးမြောက်မှသာ Transaction အတွင်းဆောင်ရွက်ထားသော Data များကို Database အပေါ် သက်ရောက်မှုရှိစေရန် ဆောင်ရွက်ဖို့လိုအပ်ပါတယ်

တစ်ခုခုအမှားဖြစ်ခဲ့ရင် Transaction အားလုံးကို မပြုလုပ်ခင် နေရာကို ပြန်ပို့နိုင်ရမယ်

Consistency

Database System တစ်ခုဟာ မှန်ကန်တိကျဖို့လိုအပ်တယ်

Transaction တစ်ခုရဲ့ မစတင်ခင် အနေအထားနဲ့ ပြီးဆုံးချိန် အနေအထားတွေမှာ Data တွေဟာ မှန်ကန်တိကျနေဖို့ လိုအပ်တယ်

အကြောင်း တစ်ခုခုကြောင့် မစခင် အနေအထားနှင့် ပြီးဆုံးချိန် အနေအထားတို့ဟာ မှားယွင်းနေလို့မရပါဘူး

Isolation

Target Data တစ်ခုအပေါ် Transaction တစ်ခု လုပ်ဆောင် နေစဉ်မှာ အခြားသော Transaction မှ ကြားဖြတ်လုပ်ဆောင် မည်ဆိုပါက အမှားအယွင်းများကို ဖြစ်စေနိုင်ပါတယ်

ထို့ကြောင့် Transaction တစ်ခု လုပ်ဆောင်နေချိန်တွေမှာ အခြား Transaction တွေနဲ့ သီးခြားရှိနေစေဖို့ လိုအပ်တယ်

Transaction တစ်ခုအသုံးပြုနေသော Data များအား၊ ၎င်း Transaction မပြီးဆုံးသေးခင် အခြား တစ်ခုမှ မမြင်နိုင်အောင်၊ အသုံးမပြုနိုင်အောင် ပြုလုပ်ထားဖို့ လိုအပ်တယ်

Durability

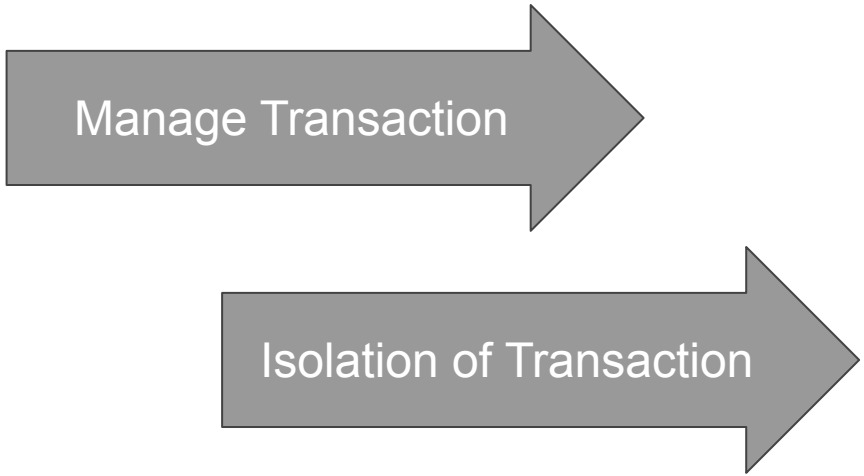
Transition တစ်ခုကနေ Commit လုပ်လိုက်တဲ့ အချက်အလက်တွေဟာ တည်မြဲနေဖို့လိုအပ်ပါတယ်

ဘယ်လိုအကြောင်းနဲ့ပဲ ကြုံကြုံ လုပ်ထားပြီးတဲ့ ဒေတာတွေဟာ ပျောက်ပျက်သွားလို့ မဖြစ်ပါဘူး

နောက်ဆုံး System က Crash ဖြစ်သွားမယ်ဆိုရင်တောင်မှ လုပ်ဆောင်ထားပြီးသား အချက်အလက်တွေဟာ တည်မြဲနေဖို့ လိုအပ်ပါတယ်

Transaction Management

Transaction Management



Manage Transaction

Transaction

Transaction Management

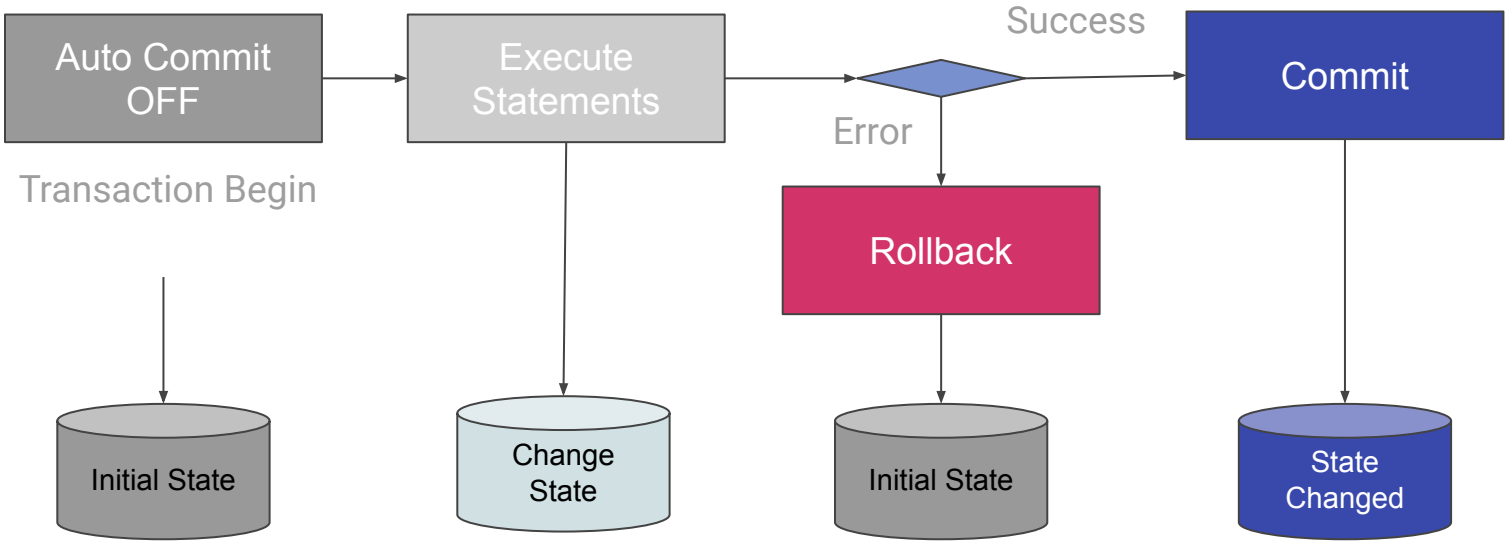
Using Save Point

Connection Methods

Transaction Management

- Connection Object တွေဟာ Default အတိုင်းဆိုရင် Auto Commit ဖြစ်နေပြီး SQL Statement ကို Execute လုပ်လိုက်တာနဲ့ Database ကို Commit လုပ်မှာ ဖြစ်ပါတယ်
- Connection ရဲ့ Auto Commit Mode ကို Off လုပ်ပြီး SQL Execution တွေ အားလုံးအောင်မြင်စွာပြီးဆုံးပြီဆိုမှ Commit လုပ်နိုင်ပါတယ်
- Auto Commit Mode ကို OFF လုပ်ထားတယ် ဆိုရင် အကြောင်းတစ်မျိုးမျိုးကြောင့် Error ဖြစ်ခဲ့ပြီဆိုပါက Rollback လုပ်နိုင်မှာ ဖြစ်ပါတယ်
- Rollback လုပ်လိုက်ရင် Transaction ထဲမှာ လုပ်ဆောင်ခဲ့တဲ့ SQL Execution အားလုံးကို Cancel လုပ်ပြစ်နိုင်ပြီး Transaction မစခင်အနေအထားကို ပြန်လည်ရောက်ရှိစေမှာ ဖြစ်ပါတယ်

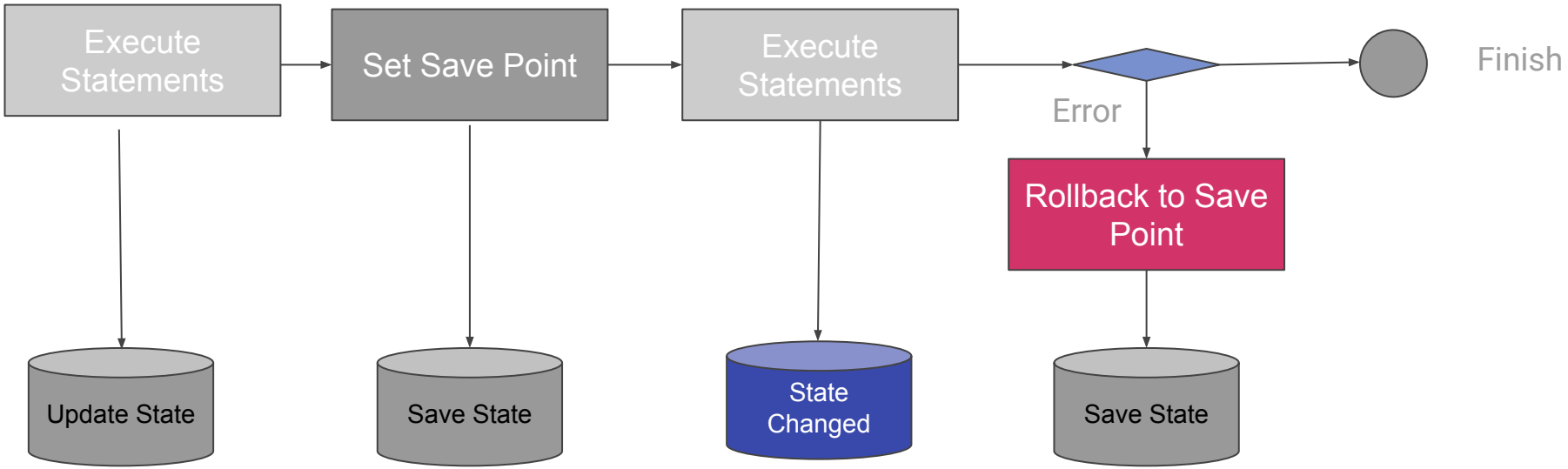
Transaction with commit & rollback



Using Save Point

- Commit & Rollback လုပ်ပြီး Transaction ကို ဆောင်ရွက်နိုင်သလို Save Point ကို အသုံးပြုပြီးလဲ Transaction Management ကို ဆောင်ရွက်နိုင်ပါတယ်
- Connection ကနေ Save Point ကို Crreate လုပ်ပြီး အကြောင်းတစ်မျိုးမျိုးကြောင့် Error ဖြစ်တဲ့ အခါမှာ Rollback လုပ်တဲ့ အခါ SavePoint နေရာအထိ Roll Back လုပ်ခိုင်းလို့ရပါတယ်
- Save Point ကိုအသုံးပြုပြီး Rollback လုပ်ပါက Save Point နောက်ပိုင်း SQL Execution တွေကိုသာ Rollback လုပ်ပြီး Save Point မတိုင်မှီ SQL Execution တွေကိုတော့ Commit လုပ်ပေးနိုင်မှာ ဖြစ်ပါတယ်

Transaction Management with Save Point



Connection

Method	Description
setAutoCommit(boolean)	Connection ရဲ့ Auto Commit Mode ကို သတ်မှတ်ရန်
getAutoCommit()	Connection ရဲ့ Auto Commit Mode ကို သိရှိရန်
commit()	Transaction ကို Commit လုပ်ရန်
rollback()	Transaction ကို Roll Back လုပ်ရန်
setSavePoint():SavePoint	Roll Back လုပ်မည့် Save Point ကိုသတ်မှတ်ရန်
setSavePoint(String):SavePoint	Roll Back လုပ်မည့် Save Point ကိုအမည်ပေးပြီးသတ်မှတ်ရန်
rollback(SavePoint)	Save Point နေရာအထိ Rollback လုပ်ရန်
setTransactionIsolation(int)	Transaction ရဲ့ Isolation Level ကို သတ်မှတ်ရန်

Isolation

Transaction

Concurrent Access

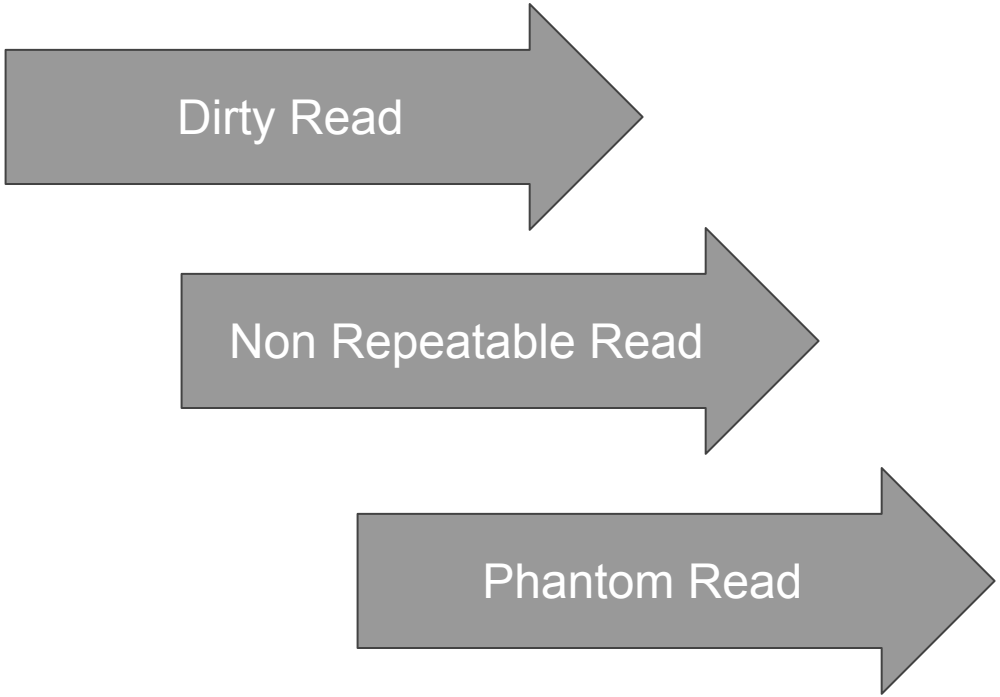
Read Problems

Isolation of Transactions

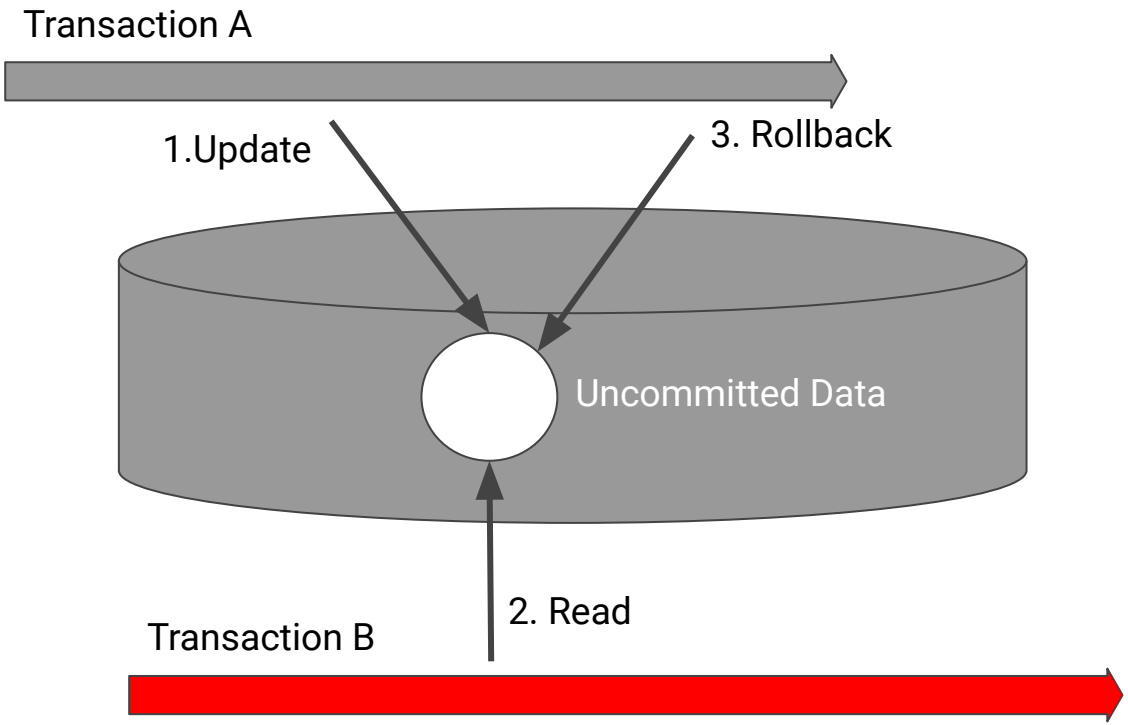
Concurrent Access & Read Problem

- RDBMS တွေဟာ Concurrent Access ကိုခွင့်ပြုထားတဲ့ အတွက် Data တစ်ခုကို တစ်ချိန်ထဲမှာ တစ်ခုထက်မကသော Transaction တွေက အသုံးပြုနေနိုင်ပါတယ်
- Transaction တစ်ခုက အသုံးပြုမည့် Data တွေကို အခြား Transaction တွေကနေ Insert, Update, Delete လုပ်လိုက် လုပ်နေနိုင်သလို၊ Commit လုပ်လိုက်၊ Rollback လုပ်လိုက်လဲ လုပ်နေနိုင်မှာ ဖြစ်ပါတယ်
- တကယ်လို့ သုံးမယ်ဆိုပြီး Read လုပ်ထားတုန်း အခြား Transaction တစ်ခုက ဝင်ပြီး Update လုပ်သွားတဲ့ အခါမှာလဲ Data တွေမမှန်တော့ဘူးဆိုတဲ့ ပြဿနာ ဖြစ်လာနိုင်ပါတယ်
- ဒါကြောင့် Transaction တစ်ခုက အသုံးပြုနေတဲ့ Data တွေကို Isolate လုပ်ထားနိုင်ဖို့ လိုအပ်ရတာ ဖြစ်ပါတယ်

Read Problems



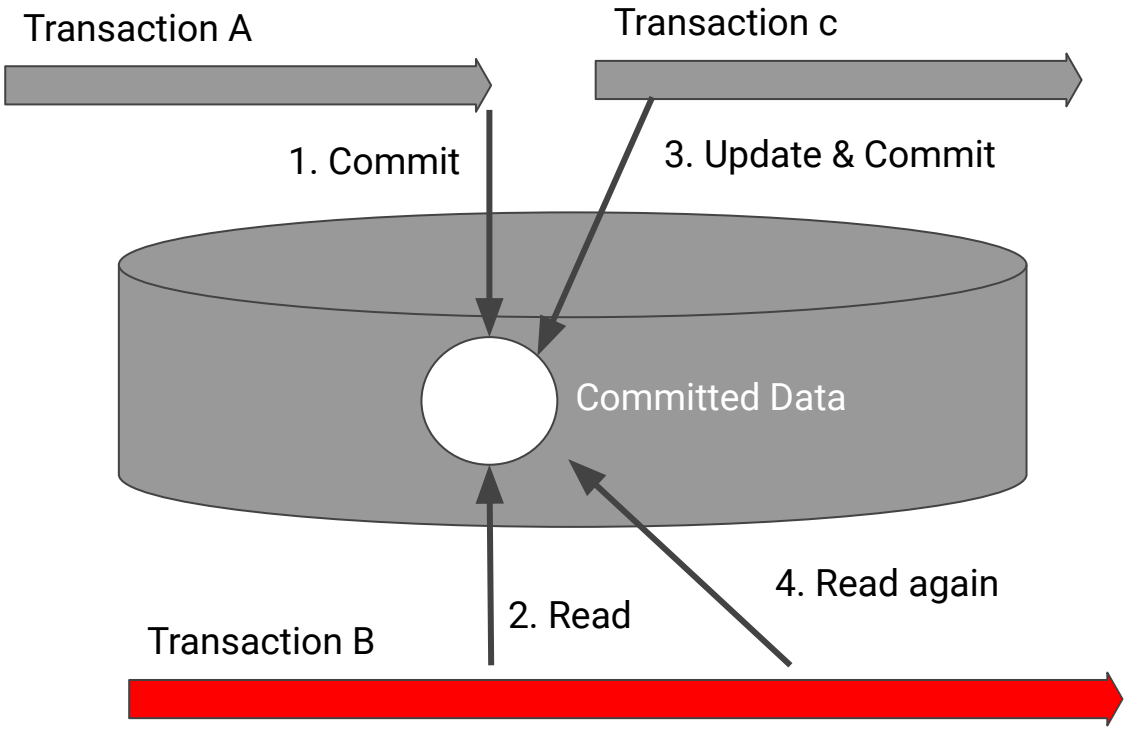
Dirty Read



Transaction တစ်ခုကနေ Commit
မလုပ်ရသေးတဲ့ Data တွေကို
အခြားသော Transaction
တစ်ခုကနေမြင်နေတဲ့
အနေအထားမျိုးမှာ
ဒီပြဿနာမျိုးကို
ဖြစ်စေတတ်ပါတယ်

Commit မလုပ်ရသေးတဲ့ Data
တွေကို Rollback လုပ်သွားရင်
အသုံးပြုနေတဲ့ Data တွေဟာ
မှန်မှာ မဟုတ်တော့ပါဘူး

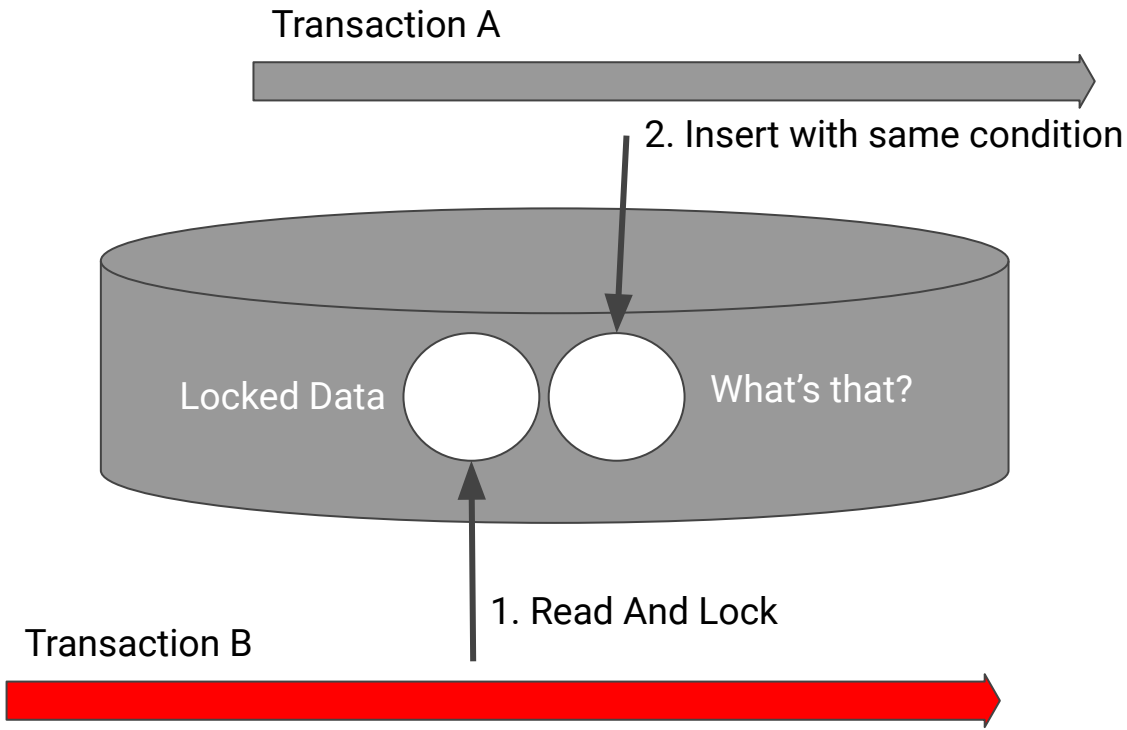
Non Repeatable Read



Transaction တစ်ခုမှာ တစ်ကြိမ်ထက်မက Data တွေကို Read လုပ်တဲ့ အခါမျိုးမှာ တန်ဖိုးတွေ မတူကြဘူးဆိုတဲ့ ပြဿနာပါ

Transaction တစ်ခုက Data တွေကို အသုံးပြုဖို့ Read လုပ်ထားတယ်၊ ပြီးတော့ အခြားသော လုပ်ဆောင်မှုတွေကို ဆက်လုန်နေချိန်၊ အခြားသော Transaction တစ်ခုကနေ သူကြည့်ထားတဲ့ Data တွေကို Update လုပ်သွားတဲ့အခါမျိုးမှာ၊ နောက်တစ်ကြိမ် ပြန်ပြီး Read လုပ်တဲ့အခါမှာ ရတဲ့ အချက်အလက်တွေနဲ့ အရင် Read လုပ်ထားတဲ့ Data တွေဟာ မတူတော့ဘူးဆိုတာ ဖြစ်သွားနိုင်ပါတယ်

Phantom Read



Transaction တစ်ခုက Condition တစ်ခုနဲ့ Data တွေကို Select လုပ်ပြီး အသုံးပြုနေစဉ်၊ အခြား Transaction တစ်ခုက အဲ့ဒီ Condition နဲ့ ကိုက်ညီတဲ့ Record တစ်ခုကို Insert လုပ်မိတယ် ဆိုတာရှိနိုင်ပါတယ်

အရင်တုန်းက မရှိခဲ့တဲ့ Data တစ်ခုဟာ အလုပ်လုပ်နေစဉ် အခြား Transaction တစ်ခုကြောင့် ရောက်ရှိလာတာကို Phantom Read လို့ ခေါ်လေ့ရှိပါတယ်

Isolation Level

Isolation Level	Description
TRANSACTION_NONE	Transaction ကို Support မလုပ်ဘူးလို့ သတ်မှတ်နိုင်ပါတယ်
TRANSACTION_READ_UNCOMMITED	Commit မလုပ်ရသေးတဲ့ Data တွေကိုလဲ Read လုပ်နိုင်တဲ့ အနေအထား
TRANSACTION_READ_COMMITED	Commit လုပ်ထားတဲ့ Data တွေကိုသာ Read လုပ်နိုင်တဲ့ အနေအထား
TRANSACTION_REPEATABLE_READ	Read လုပ်ထားတဲ့ Data တွေကို ပြောင်းမသွားအောင် ထိမ်းထားနိုင်တဲ့ အနေအထား
TRANSACTION_SERIALIZABLE	Transaction တွေကို တစ်ခုပြီးမှ တစ်ခု ပေးလုပ်တဲ့ အနေအထား

Isolation Levels & Read Problem

Isolation Level	Dirty Read	Non Repeatable Read	Phantom Read
TRANSACTION_NONE			
TRANSACTION_READ_UNCOMMITED			
TRANSACTION_READ_COMMITED	✔		
TRANSACTION_REPEATABLE_READ	✔	✔	
TRANSACTION_SERIALIZABLE	✔	✔	✔